**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ,
СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
СВЯЗИ И ИНФОРМАТИКИ»**

(МТУСИ)

Кафедра «Корпоративные информационные системы»

ЛАБОРАТОРНАЯ РАБОТА № 6

по дисциплине «Прикладные информационные системы»
на тему:

СОЗДАНИЕ ФОРМЫ, ШАБЛОНА И ПРЕДСТАВЛЕНИЯ ДЛЯ АВТОРИЗАЦИИ И РЕГИСТРАЦИИ

Выполнила: студентка группы БСТ2256
Журавлева О.А.

Студенческий билет № 3БСТ22158

Проверил: Сергеев А.Е.

Москва 2026

С О Д Е Р Ж А Н И Е

Цель работы **4**

Задачи **4**

Ход работы **5**

1 Создание формы регистрации пользователя **5**

1.1 Настройка URL для регистрации 5

1.2 Создание представления register 5

1.3 Создание шаблона registration.html 6

2 Стилизация страницы регистрации **7**

2.1 Подключение CSS 7

2.2 CSS стили для формы 7

3 Проверка уникальности пользователя **8**

3.1 Модификация представления registration 8

3.2 Отображение ошибки в шаблоне 8

4 Создание формы авторизации **9**

4.1 Настройка URL для регистрации 9

4.2 Создание представления login_view 9

4.3 Создание шаблона login.html 10

5 Стилизация страницы авторизации **11**

5.1 Подключение CSS 11

5.2 CSS стили для формы 11

6 Добавление ссылки на регистрацию 12

7 Тестирование работы формы 13

Заключение 16

Цель работы

Научиться создавать формы регистрации и авторизации пользователей в Django: получать данные из формы, валидировать их, создавать пользователей, выполнять аутентификацию и авторизацию, а также стилизовать страницы.

Задачи

1. Создать шаблон и настройте адрес для отображения страницы регистрации и авторизации.

2. Создать представление, которое обрабатывает поступающие запросы, регистрирует и авторизует новых пользователей. Сделать проверку на то, что отправленные поля непусты, введенное имя пользователя уникально, логин и пароль соответствуют одному из аккаунтов проекта.

3. Сверстать страницу в соответствии с макетом lab6_registration_form.

4. Добавить в шапку страниц всех записей и страниц для определенных статей ссылку на регистрацию в верхнем правом углу (стиль ссылке сделать точно такой же, как и ссылки “Все статьи” на собственных страницах постов).

Ход работы

1 Создание формы регистрации пользователя

Перед выполнением лабораторной работы в папке lab6 создадим копию проекта blog из лабораторной работы 5.

1.1 Настройка URL для регистрации

В файле urls.py добавим маршрут:

```
from django.contrib import admin
from django.urls import path, re_path
from articles.views import archive, get_article, create_post, register, login_view

urlpatterns = [
    path('', archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)$', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
    path('register/', register, name='register'),
    path('login/', login_view, name='login'),
    path('admin/', admin.site.urls),
]
```

1.2 Создание представления register

В файле views.py добавим функцию register.

```
def register(request):
    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        email = request.POST.get("email", "").strip()
        password = request.POST.get("password", "").strip()

        errors = {}

        if not username:
            errors["username"] = "Введите имя пользователя"
        if not password:
            errors["password"] = "Введите пароль"

        # Проверка уникальности
        if username:
            try:
                User.objects.get(username=username)
                errors["username_exists"] = "Пользователь уже существует"
            except User.DoesNotExist:
                pass

        if errors:
            return render(request, 'registration.html', {
                'form': {
                    'username': username,
                    'email': email,
```

```
                    'errors': errors
                }
            })
        else:
            User.objects.create_user(username, email, password)
            return redirect('login')

    return render(request, 'registration.html', {'form': {}})
```

1.3 Создание шаблона registration.html

В папке articles/templates/ создадим файл registration.html:

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Регистрация</title>
    <link rel="stylesheet" href="{% static 'css/registration.css' %}">
</head>
<body class="auth-page">
    <div class="content">
        <h1>Регистрация</h1>

        <form method="POST">
            {% csrf_token %}

            <input type="text" name="username" placeholder="Логин" value="{{
form.username }}">
            {% if form.errors.username %}
                <div class="error">{{ form.errors.username }}</div>
            {% endif %}
            {% if form.errors.username_exists %}
                <div class="error">{{ form.errors.username_exists }}</div>
            {% endif %}

            <input type="email" name="email" placeholder="Email" value="{{ form.email
}}">

            <input type="password" name="password" placeholder="Пароль">
            {% if form.errors.password %}
                <div class="error">{{ form.errors.password }}</div>
            {% endif %}

            <input type="submit" value="Зарегистрироваться">
        </form>
    </div>
</body>
</html>
```

2 Стилизация страницы регистрации

2.1 Подключение CSS

1. Создадим файл articles/static/css/ registration.css.

2. Подключим его в шаблоне create_post.html

```
    <head>
        <title>Создать статью</title>
        <link rel="stylesheet" href="{% static 'css/create_post.css' %}">
    </head>
```

2.2 CSS стили для формы

В registration.css пропишем стили в соответствии с макетом:

```
body.auth-page {
    background: #3498db;
    font-family: Arial;
    color: white;
}

.content {
    width: 400px;
    margin: 100px auto;
    background: rgba(0,0,0,0.3);
    padding: 30px;
    border-radius: 10px;
}

input {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
    border-radius: 5px;
    border: none;
}

input[type="submit"] {
    background: white;
    color: #3498db;
    font-weight: bold;
}

.error {
    color: #ffcccc;
    font-size: 14px;
}
```

3 Проверка уникальности пользователя

3.1 Модификация представления registration

В функции registration добавим проверку:

```
# Проверка уникальности
if username:
    try:
        User.objects.get(username=username)
        errors["username_exists"] = "Пользователь уже существует"
    except User.DoesNotExist:
        pass
```

3.2 Отображение ошибки в шаблоне

В registration.html добавим блок:

```
{% if form.errors.username_exists %}
    <div class="error">{{ form.errors.username_exists }}</div>
{% endif %}
```

При попытке создать существующего пользователя, возникнет сообщение об ошибке.

4 Создание формы авторизации

4.1 Настройка URL для регистрации

В файле urls.py добавим маршрут:

```
from django.contrib import admin
from django.urls import path, re_path
from articles.views import archive, get_article, create_post, register, login_view

urlpatterns = [
    path('', archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)$', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
    path('register/', register, name='register'), #форма регистрации
    path('login/', login_view, name='login'), #форма авторизации
    path('admin/', admin.site.urls),
]
```

4.2 Создание представления login_view

В файле views.py добавим функцию login_view.

```
def register(request):
    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        email = request.POST.get("email", "").strip()
        password = request.POST.get("password", "").strip()

        errors = {}

        if not username:
            errors["username"] = "Введите имя пользователя"
        if not password:
            errors["password"] = "Введите пароль"

        # Проверка уникальности
        if username:
            try:
                User.objects.get(username=username)
                errors["username_exists"] = "Пользователь уже существует"
            except User.DoesNotExist:
                pass

        if errors:
            return render(request, 'registration.html', {
                'form': {
                    'username': username,
                    'email': email,
                    'errors': errors
                }
            })
        else:
            User.objects.create_user(username, email, password)
            return redirect('login')

    return render(request, 'registration.html', {'form': {}})
```

4.3 Создание шаблона login.html

В папке articles/templates/ создадим файл login.html:

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Вход</title>
    <link rel="stylesheet" href="{% static 'css/login.css' %}">
</head>
<body class="auth-page">
    <div class="content">
        <h1>Вход</h1>

        <form method="POST">
            {% csrf_token %}

            <input type="text" name="username" placeholder="Логин" value="{{
form.username }}">
            <input type="password" name="password" placeholder="Пароль">

            {% if form.errors.auth %}
                <div class="error">{{ form.errors.auth }}</div>
            {% endif %}

            <input type="submit" value="Войти">
        </form>
    </div>
</body>
</html>
```

5 Стилизация страницы авторизации

5.1 Подключение CSS

1. Создадим файл articles/static/css/ login.css.

2. Подключим его в шаблоне create_post.html

```
    <head>
        <title>Создать статью</title>
        <link rel="stylesheet" href="{% static 'css/create_post.css' %}">
    </head>
```

5.2 CSS стили для формы

В registration.css пропишем стили в соответствии с макетом:

```
body.auth-page {
    background: #3498db;
    font-family: Arial;
    color: white;
}

.content {
    width: 400px;
    margin: 100px auto;
    background: rgba(0,0,0,0.3);
    padding: 30px;
    border-radius: 10px;
}

input {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
    border-radius: 5px;
    border: none;
}

input[type="submit"] {
    background: white;
    color: #3498db;
    font-weight: bold;
}

.error {
    color: #ffcccc;
    font-size: 14px;
}
```

6 Добавление ссылки на регистрацию

В шаблонах (archive.html, article.html) добавим:

```
<div class="top-menu">
    <a href="{% url 'register' %}">Регистрация</a>
</div>
```

7 Тестирование работы формы

1. Запустим сервер разработки и перейдём на страницу http://127.0.0.1:8000/register/ (рис. 1):

```
python manage.py runserver
```

2. Проверка работы формы (рис. 2-7):

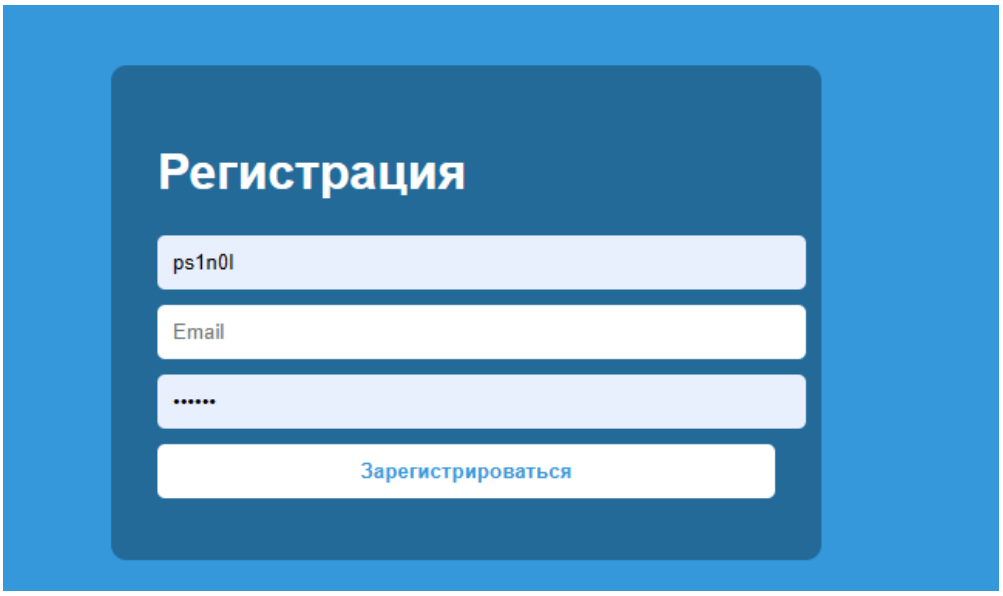

Рисунок 1 – страница http://127.0.0.1:8000/register/.

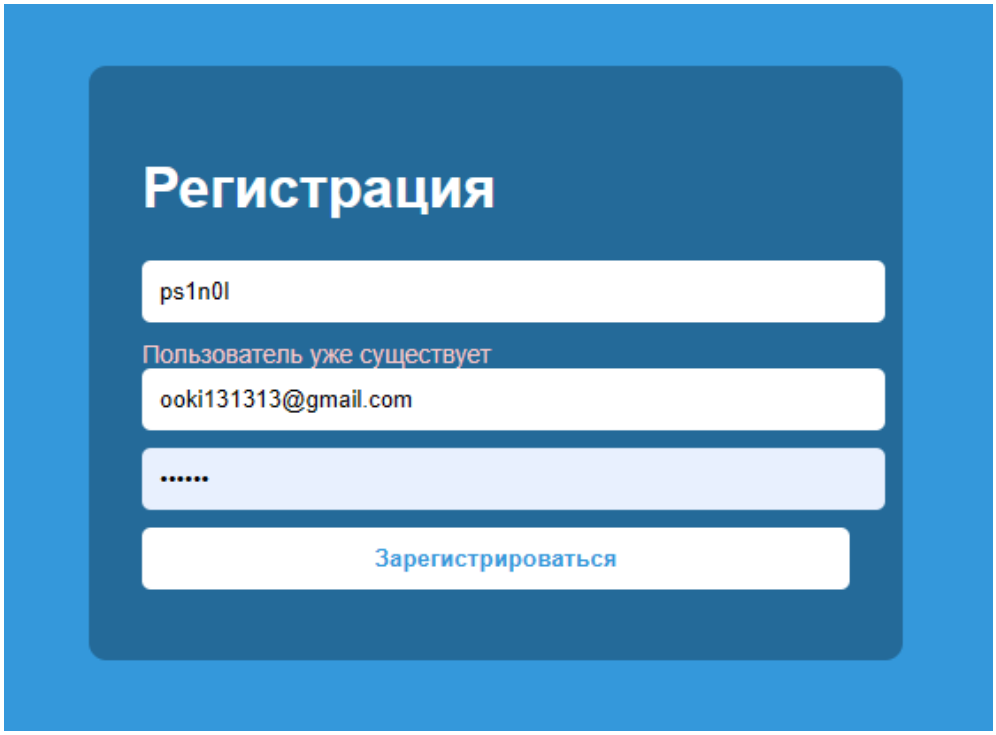

Рисунок 2 – попытка регистрации существующего пользователя.

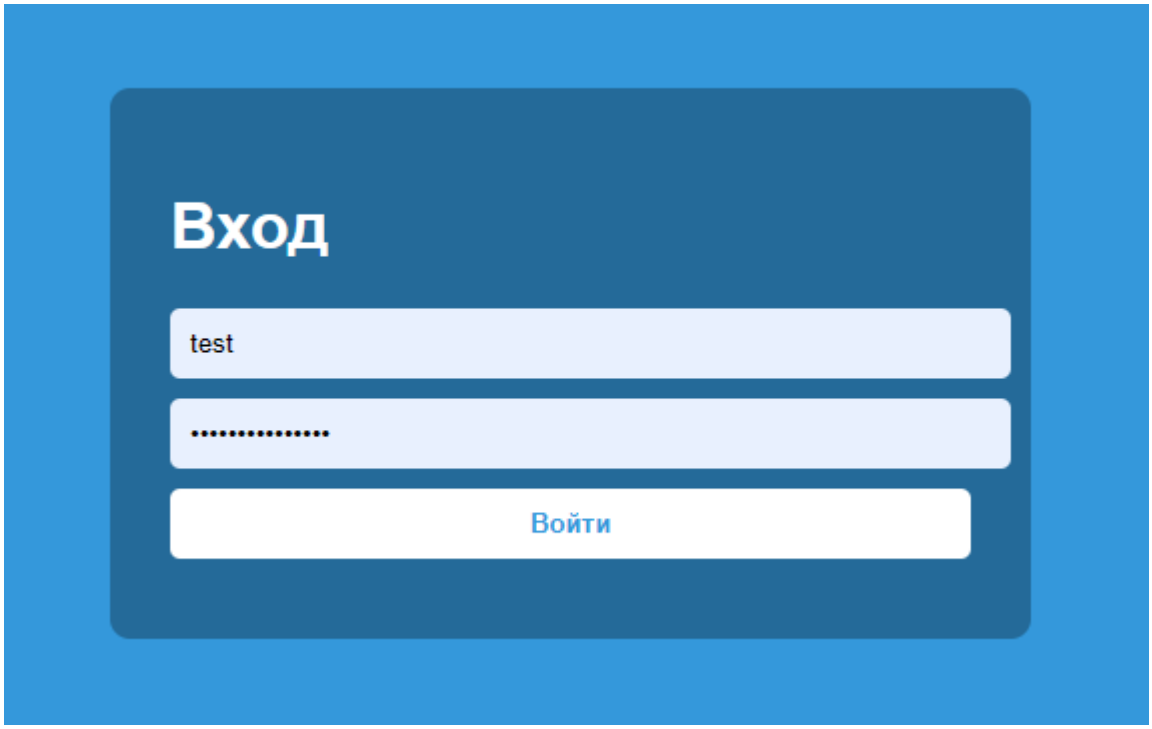

Рисунок 3 – страница http://127.0.0.1:8000/login/ после регистрации.

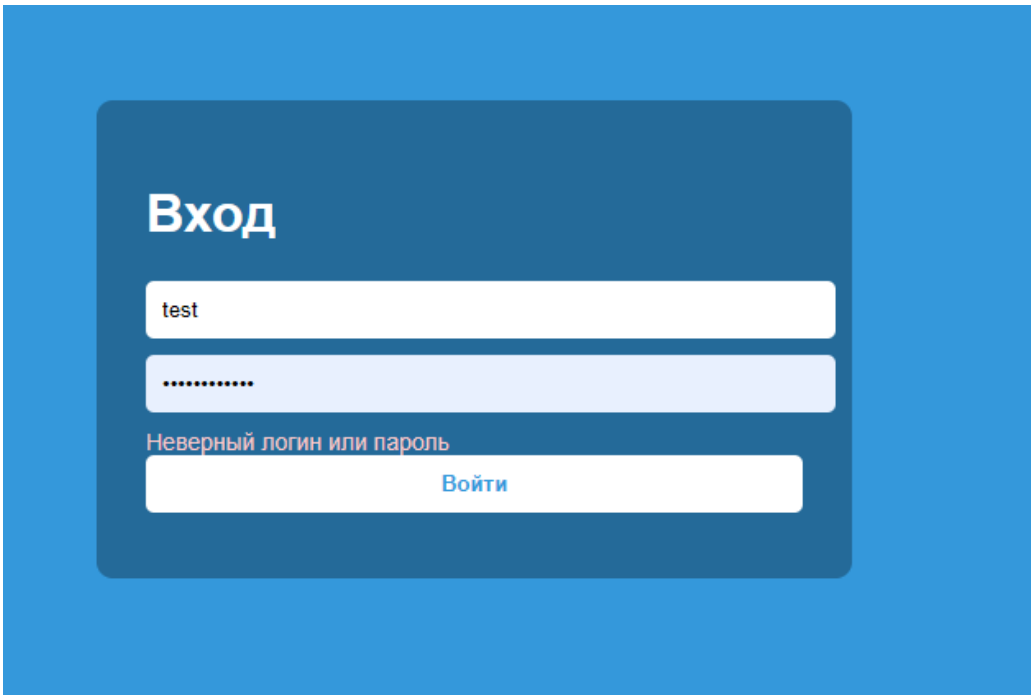

Рисунок 4 – попытка входа с неправильными учетными данными.

Рисунок 5 – страница после авторизации.

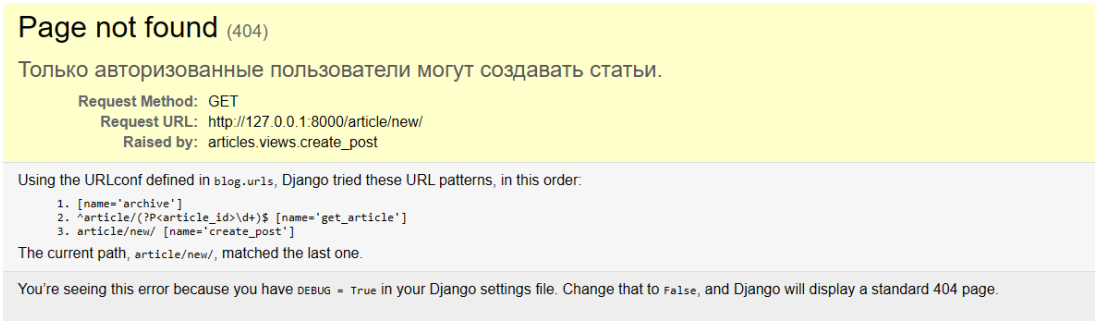

Рисунок 6 – переход на страницу без авторизации.

Заключение

В ходе лабораторной работы реализована система регистрации и авторизации пользователей в Django. Созданы представления register и login_view с обработкой GET и POST запросов. Реализована проверка уникальности имени пользователя и корректности введённых данных. Использованы функции authenticate и login для входа пользователя. Созданы HTML-шаблоны и подключены CSS-стили. Добавлены ссылки на регистрацию в интерфейс. Проведено тестирование — регистрация и авторизация работают корректно.